

N100 Financial Intelligence Platform

Analyst Guide

User documentation for the Streamlit dashboard, financial analysis workflow, PDF reports, and REST API.

Project: N100 Financial Intelligence Platform

Database: nifty100.db

Dashboard: Streamlit

API: FastAPI

Report generation: ReportLab

This guide explains how an analyst can start the platform, navigate the dashboard, screen companies, inspect company information, generate reports, use the API, and troubleshoot common runtime issues.

1. Platform Overview

The N100 Financial Intelligence Platform is a financial analytics application built around a SQLite database containing structured information for the project's Nifty 100 company universe.

Core Components

Component	Purpose
SQLite database	Stores structured company and financial data.
ETL layer	Loads, normalizes, validates, and prepares data.
Analytics layer	Calculates financial metrics and analytical outputs.
Streamlit dashboard	Provides the interactive analyst interface.
FastAPI service	Exposes financial information through REST endpoints.
Reports module	Generates PDF financial reports and tearsheets.

Project Structure

```
src/  
analytics/  
api/  
dashboard/  
etl/  
nlp/  
reports/  
screener/  
  
tests/  
api/  
db/  
dq/  
etl/  
integration/  
kpi/  
performance/  
screener/  
  
docs/  
output/  
reports/  
nifty100.db
```

Primary Services

The dashboard runs on port 8501 and the FastAPI application runs on port 8000 during normal local development.

2. Environment and Startup

Activate the Virtual Environment

```
.venv\Scripts\Activate.ps1
```

Start FastAPI

```
uvicorn src.api.main:app --host 127.0.0.1 --port 8000
```

After startup, the API is available locally at **http://127.0.0.1:8000**. FastAPI documentation is available through the application's documentation endpoint.

Start Streamlit

```
streamlit run src\dashboard\app.py --server.port 8501
```

The Streamlit dashboard is available at **http://localhost:8501**.

Normal Two-Service Setup

```
Terminal 1
uvicorn src.api.main:app --host 127.0.0.1 --port 8000
```

```
Terminal 2
streamlit run src\dashboard\app.py --server.port 8501
```

Expected Ports

Service	Port	Purpose
FastAPI	8000	REST API
Streamlit	8501	Interactive dashboard

3. Dashboard Navigation

The Streamlit application is organized into multiple dashboard pages. The analyst can use the navigation controls to move between company analysis, screening, peer analysis, trends, sector analysis, capital allocation, and reporting.

Page	Primary purpose
Home	Platform overview and key market/company information.
Profile	Detailed analysis of an individual company.
Screener	Filter companies using financial criteria.
Peers	Compare companies within peer groups.
Trends	Analyze financial trends across periods.
Sectors	Review sector-level information.
Capital	Analyze capital allocation information.
Reports	Generate and access financial reports.

Navigation Workflow



For company-level analysis, an analyst will typically begin with the Screener or Profile page. Screening can identify companies matching selected criteria, after which the Profile page can be used for detailed inspection.

4. Home and Company Profile

Home Screen

The Home screen provides the main entry point to the dashboard. It presents the platform's high-level financial information and serves as the starting point for navigating to detailed analysis.

Company Profile

The Company Profile page is designed for detailed analysis of an individual company. Select a company/ticker through the available dashboard controls to inspect its stored information.

Typical Profile Information

Information	Description
Company identity	Company name and ticker-related information.
Sector	Broad and sub-sector classification.
Financial ratios	Stored profitability, leverage and efficiency metrics.
Profit & loss	Historical income statement information.
Balance sheet	Historical balance-sheet information.
Cash flow	Operating, investing and financing cash-flow information.
Market information	Stored market-cap and valuation information.

Profile Usage Workflow

1. Open Company Profile.
2. Select the required company.
3. Review company and sector information.
4. Inspect financial ratios.
5. Review historical financial statements.
6. Compare relevant metrics across periods.
7. Use Reports when a PDF output is required.

5. Screener

The Screener page is used to filter the company universe according to financial conditions. It allows an analyst to narrow the dataset before performing detailed company-level analysis.

Example Screening Criterion

```
min_roe = 15
```

The corresponding API request used during Day 43 testing was:

```
GET /api/v1/screener?min_roe=15
```

The tested request returned HTTP 200 and produced a response containing 54 companies in the tested project database.

Recommended Screening Workflow

1. Open Screener.
2. Set the required financial filters.
3. Run the screening operation.
4. Review the returned companies.
5. Select companies for deeper Profile analysis.
6. Generate reports where required.

Interpretation

Screening results represent the output of the selected filters. They should be interpreted together with the underlying financial data and the other available dashboard analysis rather than as a standalone conclusion.

6. Peers, Trends and Sectors

Peer Analysis

The Peers page supports comparison between companies that belong to relevant peer groups. Analysts can use peer information to understand relative financial characteristics.

Trends

The Trends page is intended for examining changes in financial metrics across available periods. Historical values should be read with attention to the years actually present in the database.

Sector Analysis

The Sectors page organizes companies using the sector information stored in the database. It can be used to inspect broad-sector and sub-sector classifications and related financial information.

Screen	Analyst question
Peers	How do selected companies compare with their peers?
Trends	How have selected metrics changed over time?
Sectors	How are companies distributed across sectors?

Important Data Consideration

The platform's project documentation identifies certain supporting datasets as simulated according to the project plan. Such data should not be represented as live market data.

7. Capital Allocation and Reports

Capital Allocation

The Capital page provides access to capital-allocation-related analysis available in the platform. Analysts can use the page alongside company financial statements and ratios for context.

Reports

The Reports page provides the interface for generating financial report outputs. The platform includes a report/tearsheet module under **src/reports/**.

PDF Tearsheets

A tearsheet is a compact PDF representation of relevant company information. The exact available fields depend on the current report implementation.

Typical Workflow

1. Open Reports.
2. Select the required company or report option.
3. Generate the report.
4. Wait for PDF generation to complete.
5. Open/save the generated PDF.
6. Verify the company and reporting information before use.

Report Code

```
src/reports/  
batch_reports.py  
portfolio_summary.py  
tearsheet.py
```


8. REST API Usage

The platform provides a FastAPI REST service. The API uses the prefix **/api/v1** for versioned endpoints.

API Base URL

```
http://127.0.0.1:8000/api/v1
```

Screener Example

```
curl "http://127.0.0.1:8000/api/v1/screener?min_roe=15"
```

Health Check

```
curl "http://127.0.0.1:8000/"
```

PowerShell Alternative

```
Invoke-RestMethod "http://127.0.0.1:8000/api/v1/screener?min_roe=15"
```

Python Requests Example

```
import requests

response = requests.get(
    "http://127.0.0.1:8000/api/v1/screener",
    params={"min_roe": 15},
    timeout=5,
)

print(response.status_code)
print(response.json())
```

API Documentation

When FastAPI is running, use the application's documentation endpoint to inspect available routes, parameters, and schemas.

9. Testing and Performance

Test Suite

```
pytest tests -v
```

Performance Tests

```
pytest tests\performance\test_screener_load.py -v -s
```

```
pytest tests\performance\test_profile_performance.py -v -s
```

Day 43 Screener Result

Metric	Measured
Concurrent requests	10
Successful requests	10/10
Total time	0.0769 sec
Maximum request	0.0753 sec
Average request	0.0583 sec

Company Profile Results

Ticker	Load time
TCS	0.0119 sec
HDFCBANK	0.0062 sec
HINDUNILVR	0.0061 sec
RELIANCE	0.0059 sec
SUNPHARMA	0.0054 sec

All five tested Company Profile data-loading measurements were below the Day 43 target of 3 seconds.

10. Troubleshooting

FastAPI Will Not Start

Check whether another process is already using port 8000. Stop the conflicting process or start FastAPI on an available port and update the client configuration accordingly.

```
netstat -ano | findstr :8000
```

Streamlit Will Not Start

Check port 8501:

```
netstat -ano | findstr :8501
```

Database Not Found

Ensure that **nifty100.db** exists in the expected project location and that the application is being started from the project root.

API Returns an Error

1. Confirm FastAPI is running.
2. Check the API URL.
3. Check the endpoint path.
4. Inspect the FastAPI terminal output.
5. Test the endpoint through the API documentation.

Streamlit Warnings During Pytest

The Day 43 Company Profile performance tests produced Streamlit warnings such as 'No runtime found, using MemoryCacheStorageManager' and 'missing ScriptRunContext'. These occurred because Streamlit functions were exercised from pytest/bare mode. They did not cause the performance tests to fail.

11. Analyst Quick Reference

Startup Checklist

1. Activate `.venv`.
2. Confirm `nifty100.db` exists.
3. Start FastAPI on port 8000.
4. Start Streamlit on port 8501.
5. Open the Streamlit dashboard.
6. Verify the required dashboard page loads.

Analysis Checklist

1. Use Screener to identify companies matching criteria.
2. Open Company Profile for detailed information.
3. Review ratios and historical statements.
4. Use Peers for comparative analysis.
5. Use Trends for historical changes.
6. Use Sectors for sector-level context.
7. Use Capital for capital-allocation analysis.
8. Generate a PDF tearsheet/report when required.

API Checklist

1. Confirm FastAPI is running.
2. Use `/api/v1` endpoints.
3. Check HTTP status codes.
4. Validate returned JSON.
5. Use the API documentation when exploring endpoints.

Documentation and Quality Checklist

Item	Location
Analyst guide	docs/analyst_guide.pdf
API collection	docs/N100_API.postman_collection.json
OpenAPI specification	docs/openapi.json
Performance notes	output/perf_notes.md
Test suite	tests/
Source code	src/

End of Analyst Guide